Drag Gesture Example (Flutter)

A simple Flutter example demonstrating how to **drag a widget freely on the screen** using `GestureDetector` and `onPanUpdate`. The widget position updates in real time while staying within screen bounds.

✨Features

- Draggable widget using touch gestures
- Smooth real-time movement
- Keeps the widget **inside screen boundaries**
- Uses `Stack` + `Positioned` for free positioning
- Simple and beginner-friendly

🍂Widget Overview

Widget type: `StatefulWidget`
Use case: Draggable UI elements such as floating buttons, mini-games, movable icons, or overlays

---

🦏State Variables

Variable	Type	Description
`x`	`double`	Horizontal (left) position of the widget
`y`	`double`	Vertical (top) position of the widget

These values are updated continuously while dragging.

How It Works

1. Initial Position

```
double x = 100;
double y = 150;
```

The widget starts at a fixed position inside the screen.

---

2. Gesture Detection

```
GestureDetector(
  onPanUpdate: (details) {
    setState(() {
      x += details.delta.dx;
      y += details.delta.dy;
    });
  },
)
```

- `onPanUpdate` fires continuously while dragging
- `details.delta` provides movement since the last frame

---

3. Screen Boundary Clamping

```
x = x.clamp(0.0, screenSize.width - 80);
y = y.clamp(0.0, screenSize.height - 80);
```

This ensures the widget: - Cannot move off-screen - Always remains visible

`80` represents the widget's width & height.

⛈️UI Structure

1. `Scaffold` with black background
2. `Stack` for free positioning
3. `Positioned` widget using `x` and `y`
4. `GestureDetector` to track drag gestures
5. A styled `Container` with an icon

🌳Usage Example

Simply use `DragExample` as a screen or widget:

```
MaterialApp(
  home: DragExample(),
);
```

🦠 Customization Ideas

- Change widget size and update clamp values
- Add **snap-to-edge** behavior
- Animate movement using `AnimatedPositioned`
- Add **momentum / physics** after drag end
- Restrict movement to X-axis or Y-axis only

🦗 Notes

- Clamp values must match widget size
- Frequent `setState` calls are expected during drag
- Best for lightweight widgets

🦤 Full Source Code

```
class DragExample extends StatefulWidget {
  const DragExample({super.key});

  @override
  State<DragExample> createState() => _DragExampleState();
}

class _DragExampleState extends State<DragExample> {
  double x = 100;
  double y = 150;

  @override
  Widget build(BuildContext context) {
    final screenSize = MediaQuery.of(context).size;

    return Scaffold(
      backgroundColor: Colors.black,
      body: Stack(
        children: [
          Positioned(
            left: x,
            top: y,
            child: GestureDetector(
              onPanUpdate: (details) {
                setState(() {
                  x += details.delta.dx;
                  y += details.delta.dy;
```

```
                // keep widget inside screen
                x = x.clamp(0.0, screenSize.width - 80);
                y = y.clamp(0.0, screenSize.height - 80);
              });
            },
            child: Container(
              width: 80,
              height: 80,
              decoration: BoxDecoration(
                color: Colors.blue,
                borderRadius: BorderRadius.circular(12),
              ),
              child: const Icon(
                Icons.sports_esports,
                color: Colors.white,
                size: 40,
              ),
            ),
          ),
        ),
      ],
    ),
  );
 }
}
```